

DATA 542 – Analysis of Autonomous AI Agents in Software Development: Success, Efficiency, and Evolution

TAO WANG, University of British Columbia, Canada

In this project, I show you how five different AI coding agents (Claude_Code, Copilot, Cursor, Devin, OpenAI_Codex) perform when creating pull requests on GitHub. I used a very large dataset with about 930,000 pull requests. My goal is to understand how often their pull requests are merged, how quickly they are merged, and if there are proxies that get better over time. Based on the results I got, OpenAI_Codex performed the best since it has the highest merge rate and shortest merge time, while Devin and Copilot initially performed badly but improved in the following months. I also noticed that longer pull request descriptions can sometimes be helpful for some agents, although the effect is not significant and doesn't behave consistently across agents. These patterns show that these agents do not follow the same workflow style. Some change over time, while others appear more stable.

1 Introduction

To be honest, it's still unclear whether all code written by AI agents is easy for reviewers to handle, and we don't know if different AI agents produce code of the same quality or require the same amount of time. Because these questions remain unanswered, I want to do some researches and to figure out how these AI Agents actually work in real GitHub workflows. I simply want to see whether people in the real world actually accept the PRs they submit, and whether I can see any obvious patterns from this large amount of data.

In order to find the answer, I started this project with three main questions that seemed most relevant to comparing the agents.

- **RQ1:** How do the five autonomous coding agents differ in pull-request success rate and efficiency?
- **RQ2:** Does the length of the pull-request description affect the chance of being merged, and does this effect vary across agents?
- **RQ3:** Did any of the agents improve over time (from Dec 2024 to Jul 2025)?

2 Methodology

In the following section, I go over how I handled each question.

2.1 RQ1: How do the five autonomous coding agents differ in pull-request success rate and efficiency?

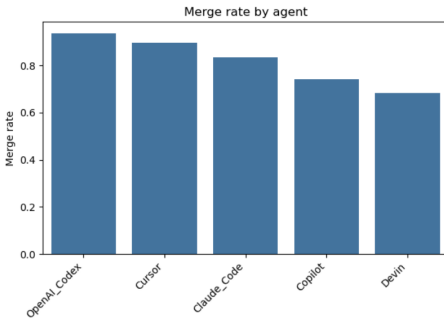
To compare the agents, I first focused on closed PRs and created two summary measures for each one:

- the merge rate (the proportion of closed PRs that were merged),
- the median time-to-merge in hours (computed only for merged PRs).

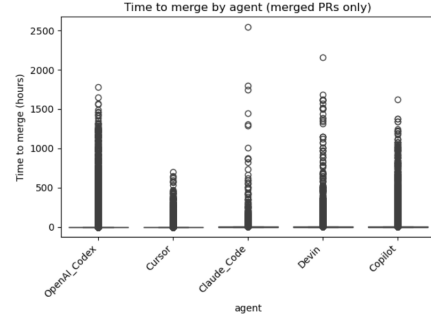
Based on what we learned in Statistics, I used a chi-square test in order to check whether the merge rates are different for different agents.

At first, I considered running a standard ANOVA for the time-to-merge values. However, after checking the distribution, I found that the dataset is obviously not normally distributed. So I switched to the Kruskal-Wallis test since it fits this dataset better.

To illustrate the findings, we plotted a bar chart of merge rates and a set of boxplots showing the distribution of time-to-merge for each agent.



(a) Merge rate by agent



(b) Time to merge (hours) by agent; merged PRs only

Fig. 1. Pull-request success rate and efficiency patterns across agents.

2.2 RQ2: Does the length of the pull-request description affect the chance of being merged, and does this effect vary across agents?

To study whether longer descriptions help PRs get merged, and whether this varies by agent, I fitted a logistic regression model:

$$\text{merged} \sim \log(\text{body_length}) * \text{agent}$$

The interaction terms allow each agent to respond differently to increases in description length.

Since regression coefficients can be hard to interpret on their own, I also divided PRs into four description-length quartiles (from Q1 shortest to Q4 longest) and compared merge rates across agents within each quartile.

2.3 RQ3: Did any of the agents improve over time (Dec 2024 – Jul 2025)?

For the temporal analysis, I extracted the year-month from each PR's creation timestamp and computed, for every agent and month:

- the number of PRs submitted,
- the merge rate for that month.

I also plotted the month-by-month merge rate curves and calculated Spearman correlations between the month index and each agent's merge rate to see if there are any trends.

3 Results

3.1 RQ1: How do the five autonomous coding agents differ in pull-request success rate and efficiency?

When I compared these agents, I found that the differences are really big. OpenAI_Codex had the highest merge rate (around 0.938), followed by Cursor and Claude_Code. Copilot's merge rate was noticeably lower (around 0.7409). Devin was even lower than Copilot (around 0.68). For time-to-merge, OpenAI_Codex and Cursor again appeared faster than the others, often finishing within minutes. Copilot generally needs more time. Because the distributions contained heavy outliers, the Kruskal-Wallis test confirmed that the differences between these agents were statistically meaningful (the p-value is less than 0.05).

So, based on the results, I found that agents with higher merge rates also tended to complete merges more quickly.

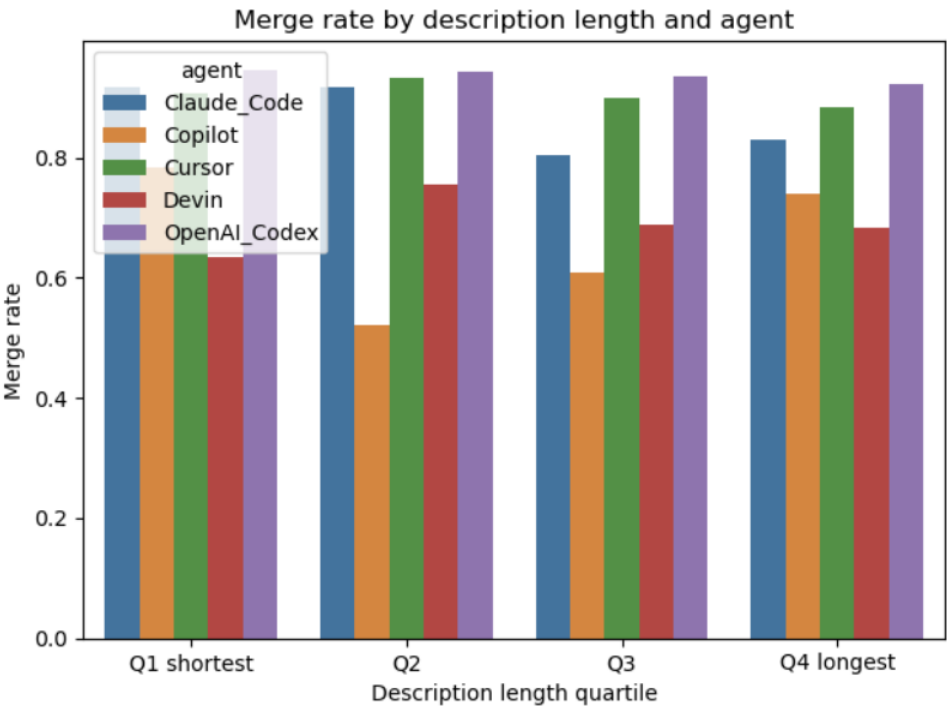


Fig. 2. Merge rate by description length and agent

3.2 RQ2: Does the length of the pull-request description affect the chance of being merged, and does this effect vary across agents?

Based on the logistic regression, there is a relationship between length and merge chance, but it's honestly pretty small. The interaction terms were actually the most useful part, and they showed me that not every agent cares about length in the same way.

For Copilot, the interaction term was positive and statistically significant, meaning that writing a longer description gave its PRs a slightly better chance of being merged. Cursor, however, showed the opposite pattern. Its negative interaction term suggests that very long descriptions did not help and may even have reduced its merge chance a little. For Devin and OpenAI_Codex, the interaction effects were small and statistically weak, implying that extra text did not change their outcomes in any meaningful way.

Since logistic regression coefficients can be difficult to interpret on their own, we also grouped PRs into four description-length quartiles and compared merge rates across agents. This visualization (Figure 2) made the differences clearer. Cursor and Claude_Code showed slightly higher merge rates in the longer-description groups (Q3–Q4), while Copilot and Devin displayed more irregular patterns. Some of this variation is likely due to uneven sample sizes across quartiles, especially for agents with fewer PRs.

The results suggest that description length plays only a minor role, and the effect depends on which agent we choose to use.

Table 1. Spearman correlation between month index and merge rate (Dec 2024 - Jul 2025)

Agent	Spearman (ρ)	P-value	Trend
OpenAI_Codex	1.00	< 0.001	Strong Upward
Copilot	0.90	0.037	Moderate Upward
Devin	0.88	0.004	Strong Upward
Claude_Code	-0.26	0.623	No Trend
Cursor	-0.13	0.788	No Trend

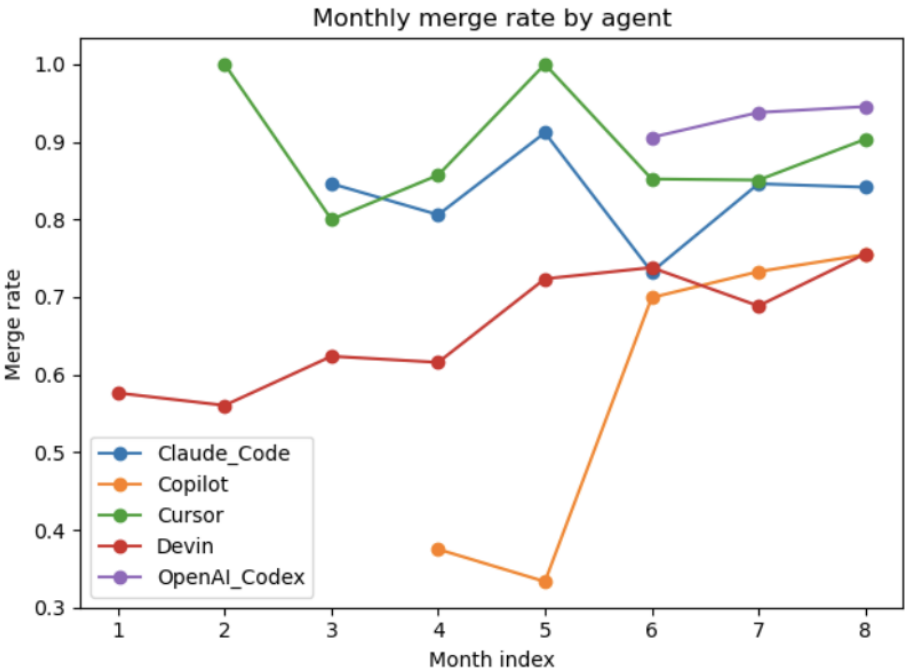


Fig. 3. Monthly merge rate by agent (Dec 2024–Jul 2025)

3.3 RQ3: Did any of the agents improve over time (Dec 2024 – Jul 2025)?

To see if the agents were actually getting better over time, I calculated the Spearman correlation for each agent. The results are summarized in Table 1.

Based on the table and the plot, OpenAI_Codex and Devin showed the most obvious improvement, with very high correlation scores. Copilot also trended up, though it was a bit less consistent statistically.

Cursor and Claude_Code didn’t really show a clear pattern in their month-to-month results. Their month-to-month merge rates became more unpredictable (sometimes up, sometimes down, very unstable), especially in months when they had only a handful of PRs. With such small sample sizes, even a couple of merges or non-merges could cause noticeable swings.

4 Discussions

Based on the results from the three questions, a few things surprised me.

First, from the overall trends, OpenAI_Codex stayed the strongest agent through the whole period. Its merge rate was highest, the merge time was shortest, and it was very stable. Before the project, I first thought Claude_Code may be the best one since I felt Claude is better at coding than ChatGPT.

Other agents like Devin, although it performed pretty poorly at the beginning, it improved quite a lot as time passed. Copilot showed a drop in the early months and then recovered again. I honestly expected the agents to behave more similarly, so this difference was a bit surprising.

I originally thought that longer descriptions would help almost every agent, but the results did not fully support that idea. The effect was not significant, and different agents reacted in different ways. Longer descriptions are a little bit helpful for some agents like Cursor; however, for agents like OpenAI_Codex, a long description is not helpful at all. After I got this result, I thought about it again and again and I finally understood why it makes sense: if the code is already clean and simple, reviewers don't really need a huge paragraph to understand it.

Another thing that influenced the results was that the number of PRs across agents is different. OpenAI_Codex created far more PRs than the others, so its month-to-month pattern looked smoother. Agents with fewer PRs showed more variation, because every merge or non-merge affected their rate more strongly. Because of this, not every shift in the graphs reflected a real change in behavior—some of it might just be randomness from small sample sizes. I think we need to be very careful if we want to use these results.

5 Limitations

There are a few limitations for this project. First, it focuses on comparing the outcomes of merged and closed PRs without examining code content or reviewer comments. In fact, examining code content and reviewer comments are very helpful for us to know what is the reasons for people to reject PRs.

Second, this dataset covers too short a timeframe. It only covers eight months! We can not capture long-term trends just based on this dataset.

More seriously, I found there are some extreme outliers in the merge times, which may reflect delays specific to certain repositories or other factors that have nothing to do with the agents themselves. Therefore, in order to get more reliable conclusions, we still need to have a dataset covering a longer timeframe and we need to do more research.

6 Conclusion

Basically, the results show that different AI coding agents don't perform the same when it comes to PR reviews on GitHub.

OpenAI_Codex performs best in terms of merge success and shows the clearest pattern of improvement. Description length helps to some degree, but mainly for specific agents such as Cursor and Copilot. Trend analysis also hints that several agents may be getting better over time.

These findings may be useful for developers or team considering which AI tools fit their workflows. Future work could explore reviewer behavior, code quality, or repository-level differences.

7 Project Details

7.1 Team Roles

Tao Wang: I am the Sole author. I do everything: research question formulation, data cleaning, Python coding (using pandas,scipy,statsmodels), statistical analysis, visualization, and report writing.

7.2 GitHub Repository

- GitHub Repository: https://github.com/UBCTAO/DATA542_FinalProject
- Dataset: HuggingFace AIDev Dataset (<https://huggingface.co/datasets/hao-li/AIDev>)

7.3 Generative AI Usage Statement

This is my first time writing a project by using LaTeX, and I ran into several LaTeX errors during the writing process (for example, issues with inserting figures and formatting). I used GenAI tools such as ChatGPT 5.1 and Gemini 3 pro to help me understand and fix these LaTeX formatting errors. I also used Grammarly to correct grammatical errors and refine and improve the sentence structure of my original draft to ensure clarity (without changing the core findings or logic of the report).

References

- [1] Hao Li, et al. 2024. *AIDev: A Dataset of AI-Generated Pull Requests*. HuggingFace. Available at: <https://huggingface.co/datasets/hao-li/AIDev>.